

2.4 Numeric types: Floating-point ^

Settings

Beta features

floating-point number

A floating-point number is a real number, like 98.6, 0.0001, or -666.

float

Thus, float is a data type for floating-point numbers.

floating-point literal

A floating-point literal is written with the fractional part even if that or 99.0.

Figure 2.4.1: A program using float-type variables.

The following program reads in a floating-point value from a user and calculates the time to drive the built-in function `float()` when reading the input to convert the input string into a float.

Note that `print` handles floating-point numbers straightforwardly.

```
miles = float(input("Enter a distance in miles: "))
hours_to_fly = miles / 500.0
hours_to_drive = miles / 60.0

print(miles, "miles would take:")
print(hours_to_fly, "hours to fly")
print(hours_to_drive, "hours to drive")
```

```
Enter a distance in miles: 450
450.0 miles would take:
0.9 hours to fly
7.5 hours to drive
...
Enter a distance in miles: 1800
1800.0 miles would take:
3.6 hours to fly
30.0 hours to drive
```

scientific notation

A floating-point literal using scientific notation is written using an "10 exponent, as in `6.02e23` to represent 6.02×10^{23} .

Try 2.4.1: Energy to mass conversion.

Albert Einstein's equation $E = mc^2$ is likely the most widely known mathematical formula. The equivalence, which states that the mass (amount of matter) m of a body is directly related to the energy connected via a constant value c^2 , the speed of light squared. The significance of the equation is energy, (and theoretically, energy back to matter). The mass-energy equivalence equation can be used in nuclear reactions, such as nuclear fission or nuclear fusion, which form the basis of modern nuclear power plants.

The following program reads in a mass in kilograms and prints the amount of energy stored in the equivalent numbers of AA batteries and tons of TNT.



▶ Run

```
1  c_meters_per_sec = 299792458 # Speed of light (m/s)
2  joules_per_AA_battery = 4320.5 # Nickel-Cadmium AA batteries
3  joules_per_TNT_ton = 4.184e9
4
5  # Read in a floating-point number from the user
6  mass_kg = float(input())
7
8  # Compute E = mc^2.
9  energy_joules = mass_kg * (c_meters_per_sec**2) # E = mc^2
10 print("Total energy released:", energy_joules, "Joules")
11
12 # Calculate equivalent number of AA and tons of TNT.
13 num_AA_batteries = energy_joules / joules_per_AA_battery
14 num_TNT_tons = energy_joules / joules_per_TNT_ton
15
16 print("Which is as much energy as:")
17 print("  ", num_AA_batteries, "AA batteries")
18 print("  ", num_TNT_tons, "tons of TNT")
```

CONSOLE

OverflowError

Assigning a floating-point value outside of this range generates an

Overflow

Overflow occurs when a value is too large to be stored in the mem

interpreter.

Figure 2.4.2: Float can overflow.

```
print("2.0 to the power of 256 =", 2.0**256)
print("2.0 to the power of 512 =", 2.0**512)
print("2.0 to the power of 1024 =", 2.0**1024)
```

```
2.0 to the power of 256 = 1.15792089237e+77
2.0 to the power of 512 = 1.34078079299e+154
2.0 to the power of 1024 =
Traceback (most recent call last):
  File "<stdin>", line 3, in <module>
OverflowError: (34, 'Result too large')
```

PARTICIPATION ACTIVITY

2.4.3: Reducing the output of Pi.

Start



2x speed

```
import math

print("Default output of Pi:", math.pi)
print("Pi reduced to 4 digits after the decimal:", end="")
print(f"{math.pi:.4f}")
```

Default output of Pi: 3.141592653589793
Pi reduced to 4 digits after the decimal: 3.1416

Captions ^

1. The mathematical constant Pi (π) is irrational. Pi is a floating-point number whose digits are infinite and non-repeating. The math module defines the constant pi with the value of Pi.
2. Though Python does not attempt to output the full value of Pi, Python outputs 15 digits after the decimal.
3. `print(f"{math.pi:.4f}")` outputs Pi to only four digits after the decimal. The last digit is rounded. The value of Pi remains the same.

**CHALLENGE
ACTIVITY**

2.4.1: Gallons of paint needed to paint walls.

The number of gallons of paint needed to cover a wall is equal to the wall's total area divided by 350.0, from input, representing a wall's total area. Then, the program computes num_gallons as wall_area / 350.0. Complete the program to output the number of gallons of paint needed to cover the wall, rounded to two decimal places.

[Click here for examples](#) ▼

Note: `print(f" {my_float:.Xf} ")` outputs my_float to X digits after the decimal point.

See [How to Use zyBooks](#) for info on how our automated program grader works.

763650.1443318.qx3zqy7

```
1 # Assign wall_area with a float read from input
2 wall_area = float(input())
3
4 # Compute num_gallons
5 num_gallons = wall_area / 350.0
6
7 """ Your solution goes here """
8
```

Run

View solution ▼ (Instructors only)

↓ Download student submissions ⓘ

**CHALLENGE
ACTIVITY**

2.4.2: Numeric types: Floating-point.

763650.1443318.qx3zqy7

Start

The inverse of power_input is $1 / \text{power_input}$. The following program intends to read a floating point number, output the inverse, but the code contains an error when reading from input. Find and fix the error.

Ex: If the input is 0.870, then the output is:

The inverse of power = $1 / 0.870 = 1.149$

```
1 # Modify the following line of code
2 power_input = int(input())
3
4 power_inverse = 1 / power_input
5
6 print(f"The inverse of power = 1 / {power_input:.3f} = {power_inverse:.3f}")
```

1

2

3

Check**Next level**

View solution ▼ (Instructors only)